ACADÉMIE KPS TECHNOLOGIES · BLOC DATABRICKS · NIVEAU 1

Rapport — Alpenia Lakehouse

Projet : transformer les fichiers d'Alpenia Assurances en tables fiables · Nom : **GOUBALAN Gerry Nachor** · Novembre 2026

1 Table managée ou table externe : quelle différence ?

Une table, ce sont deux choses : une **fiche** qui la décrit (son nom, ses colonnes) et les **fichiers** qui contiennent les données.

- **Table managée** : Databricks s'occupe des deux. Il choisit où ranger les fichiers. Si on supprime la table, les données sont supprimées aussi.
- **Table externe** : Databricks ne garde que la fiche. Les fichiers restent dans un espace de stockage que l'entreprise gère elle-même. Si on supprime la table, les fichiers restent.

> Image : une table managée, c'est un appartement meublé (tout part avec le bail). Une table externe, c'est un appartement où l'on apporte ses meubles (ils restent à nous quand on part).

Mon choix : table managée par défaut, car c'est plus simple. Table externe si d'autres outils doivent lire les mêmes fichiers, ou si les données doivent survivre à la table.

2 Fichier dans le volume `landing` ou table `brz_` ?

Le **fichier** déposé dans `landing` est un export brut, tel que le système l'a produit. Pour le lire, il faut connaître son « mode d'emploi » : quel séparateur, quel format de date, etc. Mal lu, il donne des données fausses sans prévenir.

La **table** `brz_` est la version rangée de ce fichier : chaque colonne a un type clair (date, nombre, texte), on peut l'interroger directement en SQL, on contrôle qui y a accès, et on garde l'historique de toutes ses modifications.

> Image : le fichier, c'est un carton de courrier reçu en vrac ; la table, c'est le classeur où ce courrier est trié et étiqueté.

3 Pourquoi une table Delta plutôt qu'un simple dossier de fichiers ?

Une table Delta, ce sont des fichiers Parquet **plus un journal** qui note chaque modification. Ce journal apporte des capacités qu'un dossier de fichiers n'a pas. J'en ai utilisé deux :

- **Revenir en arrière (time travel)** : après avoir volontairement doublé par erreur les tarifs de Paris dans `brz_pricing`, j'ai consulté la version d'avant (`VERSION AS OF`) puis remis la table dans son état correct (`RESTORE`).
- **Protection de la structure** : Delta refuse qu'on remplace une table par des données d'une forme différente (par exemple du texte devenu date), sauf si on l'autorise clairement (`overwriteSchema`). Cela évite de casser une table par accident.

Autre avantage : une écriture est « tout ou rien ». Personne ne lit jamais une table à moitié chargée.

4 Quel mode d'écriture ai-je choisi, et pourquoi ?

J'ai choisi **`overwrite` (remplacer)** : à chaque lancement, le contenu de la table est entièrement remplacé par le contenu des fichiers. Comme chaque dossier contient l'export complet, relancer le chargement donne toujours le même résultat. J'ai lancé le chargement deux fois : le nombre de lignes est resté identique. Avec `append` (ajouter), la deuxième fois aurait recopié toutes les lignes en double.

Si de nouveaux fichiers arrivaient chaque jour, tout remplacer deviendrait lent (on relirait tout l'historique chaque jour) et risqué (si on archive les anciens fichiers, leurs données disparaîtraient de la table). Il faudrait passer à un **chargement progressif** qui ne lit que les nouveaux fichiers (outils *Auto Loader* ou `COPY INTO`, qui retiennent ce qui a déjà été chargé), et utiliser `MERGE` pour mettre à jour une ligne existante au lieu de la dupliquer.

5 À quoi sert `DESCRIBE HISTORY` au quotidien ?

C'est le **carnet de bord** de la table : pour chaque modification, il indique quand elle a eu lieu, qui l'a faite, quelle opération et combien de lignes ont été écrites.

Exemple concret : un matin, les tarifs de Paris paraissent deux fois trop élevés. En regardant le carnet de bord de `brz_pricing`, on voit qu'une modification (`UPDATE`) a été faite, à quelle heure et par qui. On peut alors relire la table telle qu'elle était juste avant, puis la remettre en état avec `RESTORE`. C'est exactement ce que j'ai fait dans le projet. Chez un assureur, un chiffre faux coûte cher : cet outil permet de trouver l'origine du problème et de le corriger vite.

6 Quelle « machine » fait tourner mon travail ?

- **Notebook** : il utilise le calcul *serverless* des notebooks. Il se connecte tout seul, en quelques secondes, sans rien configurer.
- **Éditeur SQL** : il utilise un *SQL warehouse serverless*, une machine dédiée aux requêtes SQL. À la première requête, il faut l'attacher et le démarrer, ce qui prend quelques dizaines de secondes.

« Serverless » veut dire que je ne crée et ne règle aucune machine moi-même : Databricks s'en charge. Dans mon espace de travail, le catalogue créé d'office s'appelle `workspace`, et les volumes se trouvent dans le *Catalog Explorer*, sous catalogue → schéma → *Volumes*.

Résultats des contrôles

Table	Lignes attendues	Lignes obtenues (après 2 chargements)	Valeurs perdues au typage
brz_contracts	4 000	4 000	aucune
brz_clients	3 000	3 000	aucune
brz_pricing	6	6	aucune
brz_claims	24 000	24 000	aucune
brz_web_events	90 000	90 000	aucune
brz_expertises	450	450	aucune (52 dates de clôture vides = expertises encore en cours)

Ce que j'ai appris d'une erreur

Dans `03_transformations`, la table `brz_web_events` a bien été écrite, puis une erreur est apparue juste après. La raison : sur Databricks, un DataFrame n'est pas une copie des données, c'est une **recette** qui relit la table à chaque fois qu'on s'en sert. Une fois la table réécrite sans la colonne `quote`, la recette ne trouvait plus cette colonne. Solution : afficher la structure de la table écrite, et non rejouer la recette.